Cloud Computing

Lab Submission – 7

Name: Amar

SRN: PES2UG23CS052

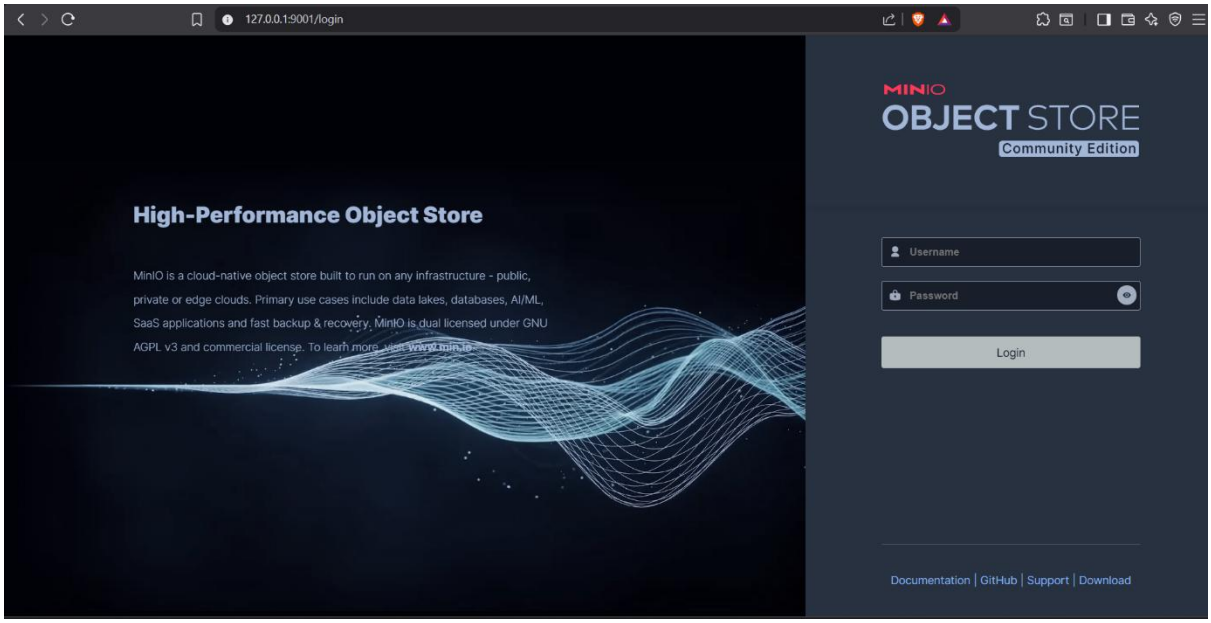
Section: A

Date: 17-03-2026

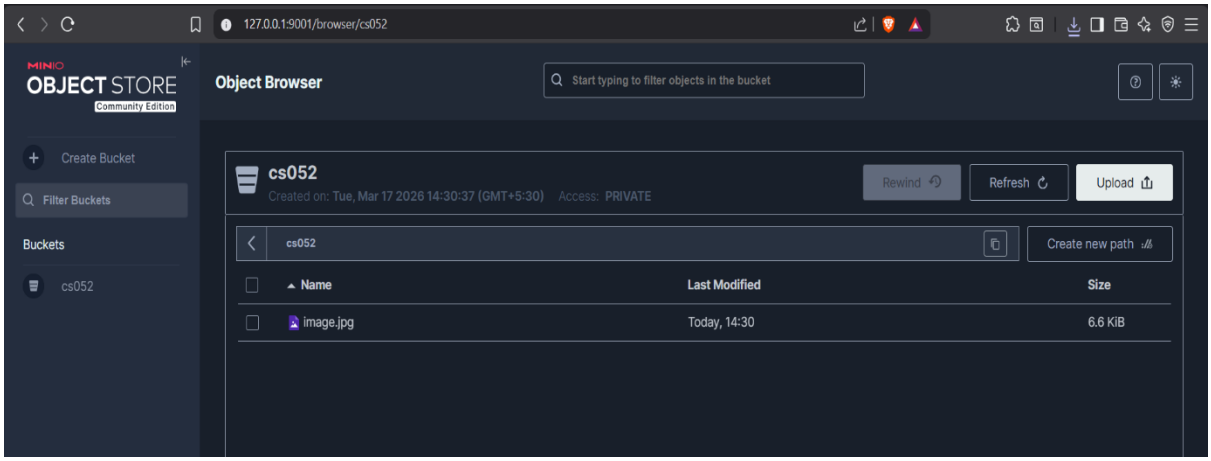
Screenshots:

Screenshot-1 browser http://localhost:500

Screenshot-2 Browser: MinIO Dashboard: http://localhost:9001



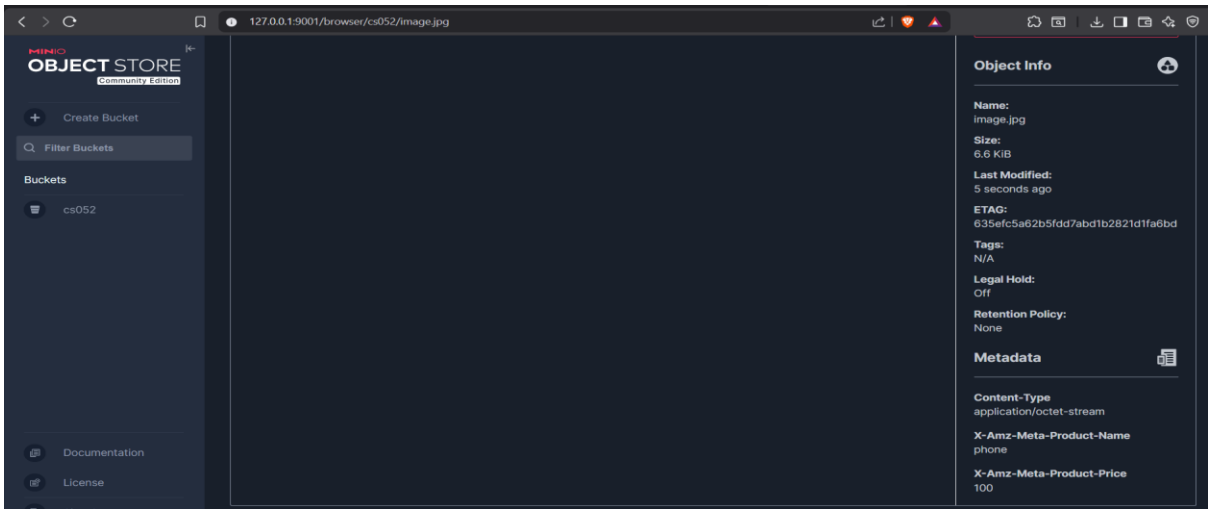
Screenshot-3 Browser: MinIO Dashboard



Screenshot-4

```
PS E:\SEM 6\CC\lab7\lab-7-storage> docker exec -it lab-7-storage-webapp-1 python -c "import sqlite3; conn = sqlite3.conn
ect('/mnt/block_volume/ecommerce.db'); cursor = conn.cursor(); cursor.execute('SELECT * FROM product'); print(cursor.fet
chall()); conn.close()"
[(1, 'Phone', 500.0, 'image.jpg'), (2, 'Phone', 500.0, 'image.jpg')]
PS E:\SEM 6\CC\lab7\lab-7-storage>
```

Screenshot-5



Screenshot-6

```
PS E:\SEM 6\CC\lab7\lab-7-storage> docker-compose stop minio
time="2026-03-17T15:44:59+05:30" level=warning msg="E:\\SEM 6\\CC\\lab7\\lab-7-storage\\docker-compose.yml: the attribut
e `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] stop 1/1
 ✔Container lab-7-storage-minio-1 Stopped                                                                          0.9s
PS E:\SEM 6\CC\lab7\lab-7-storage> curl.exe -X POST -F "name=Test" -F "price=100" -F "image=@image.jpg" http://localhost
:5000/product
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the server is overloaded or
```

Observations

Experiment-1: Access Method

1. The database file (ecommerce.db) is stored locally in the system under block storage.
2. The uploaded image is not present in the local file system.
3. The image is stored inside MinIO object storage and accessed via API.
4. Block storage behaves like a traditional disk storing structured data.
5. Object storage abstracts data access and stores unstructured data separately.

Experiment-2: Metadata vs File Headers

1. Metadata was added during file upload using MinIO API.
2. Metadata fields such as product name and price are visible in MinIO UI.
3. Metadata can be accessed without opening the actual file.
4. Block storage (database) does not support storing such metadata directly.
5. Object storage enables enrichment of data using custom metadata tags.

Experiment-3: Scaling & Fragmentation

1. Additional files were copied into block storage (my_block_data) to simulate storage usage.
2. Even after adding large files, the system continued working due to available disk space.
3. Block storage has fixed capacity and is limited by disk size.
4. When storage reaches its limit, database operations may fail with disk errors.
5. Object storage (MinIO) can scale independently and continue storing data.

Experiment-4: Recovery & High Availability

1. MinIO service was stopped and product upload was attempted.
2. Upload failed because object storage was unavailable.
3. After restarting MinIO, normal functionality was restored.
4. Web application was stopped, but database file remained intact.
5. Storage systems operate independently, ensuring data persistence and availability.